

Sesión 12: Diseño y Tematización (Implementación de Material Design 3, ThemeData, Modos Claro/Oscuro y Temas Dinámicos)

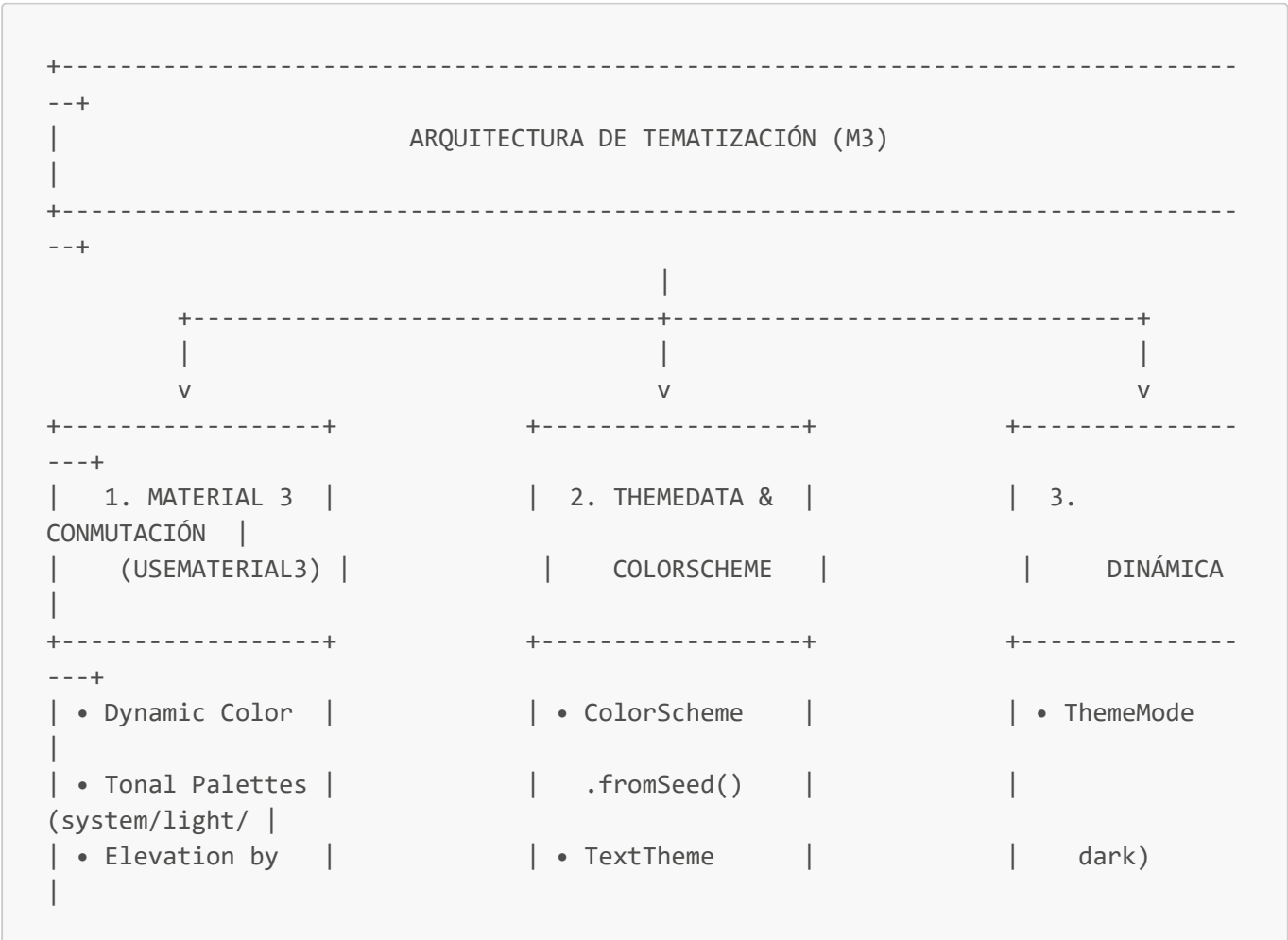
Introducción

La tematización consistente y escalable es fundamental para ofrecer una experiencia de usuario (UX) coherente, accesible y adaptada a las preferencias del usuario. En el ecosistema de Flutter, el sistema de diseño estándar está basado en **Material Design 3 (M3)**, el cual introduce un modelo de color tonal dinámico, jerarquías de elevación revisadas y esquemas de color expresivos que se adaptan automáticamente a los modos claro y oscuro del sistema operativo.

En esta sesión abordaremos la arquitectura de tematización en Flutter. Analizaremos la configuración global de **ThemeData**, el uso de la clase **ColorScheme** a partir de semillas de color (**fromSeed**), la conmutación dinámica entre el modo claro (*Light Mode*) y el modo oscuro (*Dark Mode*), y las mejores prácticas para consumir propiedades del tema en el árbol de widgets mediante **Theme.of(context)**.

Conceptos y Definiciones

El motor de tematización en Flutter desacopla las definiciones estéticas (colores, tipografía y formas) de la estructura del layout.



surface tint	• Component	• Theme.of
	Themes	(context)
+-----+	+-----+	+-----
---+		

1. Fundamentos de Material Design 3 en Flutter

- **useMaterial3: true:** Habilita las especificaciones visuales de Material Design 3. A partir de versiones recientes de Flutter, esta propiedad viene activada por defecto en `ThemeData`.
- **Esquemas Tónicos de Color (ColorScheme):** M3 genera una paleta cromática armónica completa (colores primarios, secundarios, terciarios, contenedores y de superficie) a partir de una única semilla cromática (*seed color*) mediante la función `ColorScheme.fromSeed(seedColor: ...)`.
- **Elevación y Superficies:** M3 reemplaza las sombras proyectadas pronunciadas por tonalidades de superficie (*surface tint*), variando la opacidad del color primario según la elevación del componente.

2. Configuración Global con ThemeData

El widget `MaterialApp` administra la tematización global mediante tres propiedades principales:

- **theme:** Define el objeto `ThemeData` aplicado cuando la aplicación se encuentra en modo claro (*Light Theme*).
- **darkTheme:** Define el objeto `ThemeData` aplicado cuando la aplicación se encuentra en modo oscuro (*Dark Theme*).
- **themeMode:** Controla qué tema renderizar de forma activa:
- `ThemeMode.system` (*predeterminado*): Sigue de manera automática la preferencia global definida en el sistema operativo.
- `ThemeMode.light`: Fuerza el renderizado del tema claro.
- `ThemeMode.dark`: Fuerza el renderizado del tema oscuro.

3. Consumo Programático del Tema mediante Theme.of(context)

Para garantizar que los widgets individuales mantengan coherencia visual ante cambios de tema, se evita codificar colores en duro (*hardcoding*). En su lugar, se leen los tokens del tema desde el contexto:

- **Theme.of(context).colorScheme:** Otorga acceso a los roles de color activos (ej. `primary`, `onPrimary`, `surface`, `secondaryContainer`).
- **Theme.of(context).textTheme:** Otorga acceso a las escalas tipográficas estandarizadas (ej. `headlineMedium`, `bodyLarge`, `labelSmall`).

Ejemplo Completo con Código Fuente: Aplicación con Conmutación Dinámica de Tema y Paleta M3

El siguiente ejemplo implementa una aplicación con soporte completo para Material Design 3, generación de esquemas tónicos claro/oscuro a partir del guinda institucional, y conmutación interactiva de tema en tiempo de ejecución.

Código Fuente Explicado (`lib/main.dart`)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoTematizacionApp());
}

/// Widget Raíz que administra el estado del ThemeMode
class DemoTematizacionApp extends StatefulWidget {
  const DemoTematizacionApp({super.key});

  @override
  State<DemoTematizacionApp> createState() => _DemoTematizacionAppState();
}

class _DemoTematizacionAppState extends State<DemoTematizacionApp> {
  // Estado para controlar el modo de tema activo
  ThemeMode _modoTemaActual = ThemeMode.system;

  void _cambiarModoTema(ThemeMode nuevoModo) {
    setState(() {
      _modoTemaActual = nuevoModo;
    });
  }

  @override
  Widget build(BuildContext context) {
    // Semilla cromática principal (Guinda Institucional ESCOM)
    const Color colorSemilla = Color(0xFF6B112D);

    return MaterialApp(
      title: 'ESCOM - Diseño y Tematización M3',
      debugShowCheckedModeBanner: false,

      // CONFIGURACIÓN DEL TEMA CLARO (LIGHT THEME)
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: colorSemilla,
          brightness: Brightness.light,
        ),
        appBarTheme: const AppBarTheme(
          centerTitle: true,
        ),
      ),

      // CONFIGURACIÓN DEL TEMA OSCURO (DARK THEME)
```

```

        darkTheme: ThemeData(
            useMaterial3: true,
            colorScheme: ColorScheme.fromSeed(
                seedColor: colorSemilla,
                brightness: Brightness.dark,
            ),
            appBarTheme: const AppBarTheme(
                centerTitle: true,
            ),
        ),

        // Control del modo de tema activo
        themeMode: _modoTemaActual,

        home: PantallaPanelTematizacion(
            modoTemaActual: _modoTemaActual,
            onCambiarTema: _cambiarModoTema,
        ),
    );
}

/// Pantalla Principal
class PantallaPanelTematizacion extends StatelessWidget {
    final ThemeMode modoTemaActual;
    final ValueChanged<ThemeMode> onCambiarTema;

    const PantallaPanelTematizacion({
        super.key,
        required this.modoTemaActual,
        required this.onCambiarTema,
    });

    @override
    Widget build(BuildContext context) {
        // Lectura de los tokens de color del tema activo
        final colorScheme = Theme.of(context).colorScheme;
        final textTheme = Theme.of(context).textTheme;

        return Scaffold(
            appBar: AppBar(
                title: const Text('Material Design 3 & Temas'),
                backgroundColor: colorScheme.primary,
                foregroundColor: colorScheme.onPrimary,
            ),
            body: SingleChildScrollView(
                padding: const EdgeInsets.all(20.0),
                child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                        // SECCIÓN 1: CONTROL DE MODO DE TEMA
                        Text(
                            'Selecciona el Modo de Tema',
                            style: textTheme.titleMedium?.copyWith(

```

```

        fontWeight: FontWeight.bold,
      ),
    ),
    const SizedBox(height: 12),
    SegmentedButton<ThemeMode>(
      segments: const [
        ButtonSegment(
          value: ThemeMode.system,
          label: Text('Sistema'),
          icon: Icon(Icons.brightness_auto),
        ),
        ButtonSegment(
          value: ThemeMode.light,
          label: Text('Claro'),
          icon: Icon(Icons.light_mode),
        ),
        ButtonSegment(
          value: ThemeMode.dark,
          label: Text('Oscuro'),
          icon: Icon(Icons.dark_mode),
        ),
      ],
      selected: {modoTemaActual},
      onSelectionChanged: (NuevasSelecciones) {
        onCambiarTema(NuevasSelecciones.first);
      },
    ),
    const SizedBox(height: 28),

    // SECCIÓN 2: INSPECCIÓN DE ROLES DE COLOR M3
    Text(
      'Paleta Tonal Generada (ColorScheme)',
      style: textTheme.titleMedium?.copyWith(
        fontWeight: FontWeight.bold,
      ),
    ),
    const SizedBox(height: 12),

    Row(
      children: [
        Expanded(
          child: _MuestraColor(
            nombre: 'Primary',
            colorFondo: colorScheme.primary,
            colorTexto: colorScheme.onPrimary,
          ),
        ),
        const SizedBox(width: 8),
        Expanded(
          child: _MuestraColor(
            nombre: 'Primary Container',
            colorFondo: colorScheme.primaryContainer,
            colorTexto: colorScheme.onPrimaryContainer,
          ),
        ),
      ],
    ),
  ),
),

```

```

    ),
  ],
),
const SizedBox(height: 8),
Row(
  children: [
    Expanded(
      child: _MuestraColor(
        nombre: 'Secondary',
        colorFondo: colorScheme.secondary,
        colorTexto: colorScheme.onSecondary,
      ),
    ),
  ],
),
const SizedBox(width: 8),
Expanded(
  child: _MuestraColor(
    nombre: 'Secondary Container',
    colorFondo: colorScheme.secondaryContainer,
    colorTexto: colorScheme.onSecondaryContainer,
  ),
),
],
),
const SizedBox(height: 28),

// SECCIÓN 3: TARJETA DE DEMOSTRACIÓN DE COMPONENTES
Card(
  elevation: 1,
  color: colorScheme.surfaceContainerHigh,
  child: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          'Tarjeta Adaptativa M3',
          style: textTheme.headlineSmall?.copyWith(
            color: colorScheme.onSurface,
          ),
        ),
        const SizedBox(height: 8),
        Text(
          'Esta tarjeta ajusta automáticamente sus tonos de superficie, bordes y colores de texto según el tema activo.',
          style: textTheme.bodyMedium?.copyWith(
            color: colorScheme.onSurfaceVariant,
          ),
        ),
        const SizedBox(height: 16),
        Align(
          alignment: Alignment.centerRight,
          child: FilledButton.icon(
            onPressed: () {},
            icon: const Icon(Icons.check),
          ),
        ),
      ],
    ),
  ),
),

```

```

        label: const Text('Acción M3'),
      ),
    ),
  ],
),
),
),
],
),
),
);
}
}

/// Widget auxiliar para visualizar bloques de color del Theme
class _MuestraColor extends StatelessWidget {
  final String nombre;
  final Color colorFondo;
  final Color colorTexto;

  const _MuestraColor({
    required this.nombre,
    required this.colorFondo,
    required this.colorTexto,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      height: 70,
      padding: const EdgeInsets.all(8.0),
      decoration: BoxDecoration(
        color: colorFondo,
        borderRadius: BorderRadius.circular(8),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Text(
            nombre,
            style: TextStyle(
              color: colorTexto,
              fontSize: 11,
              fontWeight: FontWeight.bold,
            ),
          ),
        ],
      ),
    );
  }
}

```

Prácticas Guiadas / Laboratorio

Práctica 1: Personalización Fina de Tipografía y Estilos de Componentes en `ThemeData`

1. Modifica la propiedad `theme` en el archivo principal para personalizar la escala tipográfica predeterminada (`TextTheme`).
2. Configura un estilo personalizado global para los botones elevados (`ElevatedButtonThemeData`):

```
theme: ThemeData(  
  useMaterial3: true,  
  colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF6B112D)),  
  elevatedButtonTheme: ElevatedButtonThemeData(  
    style: ElevatedButton.styleFrom(  
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),  
      shape: RoundedRectangleBorder(  
        borderRadius: BorderRadius.circular(12),  
      ),  
    ),  
  ),  
),
```

3. Ejecuta la aplicación para verificar cómo todos los `ElevatedButton` del proyecto heredan de manera centralizada la nueva forma redondeada sin modificar su código local.

Práctica 2: Soporte para Tema Dinámico basado en el Color del Dispositivo

1. Investiga el paquete oficial `dynamic_color` de la comunidad de Flutter.
2. Integra el widget `DynamicColorBuilder` para extraer el color tónico predeterminado del fondo de pantalla del dispositivo Android/iOS del usuario.
3. Define la propiedad `seedColor` del `ColorScheme.fromSeed()` para que responda dinámicamente al esquema de colores del sistema del usuario (*Material You*).

Resumen y Puntos Clave

- **Material Design 3 (M3):** Introduce el sistema de color tonal basado en `ColorScheme.fromSeed()`, reemplazando sombras pesadas por variaciones sutiles de tono en superficies (*surface tint*).
- **Gestión Centralizada con `ThemeData`:** Define los estilos globales de colores, fuentes y componentes para toda la aplicación en un solo lugar.
- **Soporte Nativo Claro/Oscuro:** A través de las propiedades `theme`, `darkTheme` y `themeMode` en `MaterialApp`, Flutter conmuta suavemente entre esquemas visuales.
- **Acceso Declarativo con `Theme.of(context)`:** Permite consumir `colorScheme` y `textTheme` asegurando que la interfaz reaccione dinámicamente a cualquier cambio de tema sin acoplar colores estáticos en el código.

Bibliografía y Referencias

1. **Google LLC.** (2024). *Material Design 3 - Color System*. Recuperado de:
<https://m3.material.io/styles/color/overview>
2. **Google LLC.** (2024). *ThemeData class documentation - Flutter API*. Recuperado de:
<https://api.flutter.dev/flutter/material/ThemeData-class.html>
3. **Google LLC.** (2024). *ColorScheme class documentation - Flutter API*. Recuperado de:
<https://api.flutter.dev/flutter/material/ColorScheme-class.html>
4. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega.
ISBN: 978-6075389102.